ЗВІТ. Багатопотокова структура даних, варіант №13

Демчук Дмитро · К-25 | C++17 · CMake

1. Умова задачі

Структура містить два цілі поля (m = 2), початково обидва дорівнюють нулю. Необхідні читання/запис поля 0 і 1 та отримання рядка з обох значень.

2. Частоти для варіанта

read 0 — 10% | write 0 — 10% | read 1 — 50% | write 1 — 10% | string — 20%

3. Захист даних

Застосовано два std::mutex, по одному для кожного поля. Це дозволяє незалежно читати й змінювати різні поля без зайвого спільного блокування. getText() захоплює обидва м’ютекси через std::scoped_lock, що забезпечує узгоджений знімок значень і запобігає взаємному блокуванню.

4. Методика

Згенеровано 9 файлів по 300 000 операцій: 3 за умовою, 3 рівномірних і 3 з перевагою string (96%). Окремо перевірено 1, 2 і 3 потоки. Кожен потік читає власний файл та має власний об’єкт Fields. Час виміряно через steady_clock від створення потоків до завершення join().

5. Результати виконання (мілісекунди)

Тип тесту	1 потік	2 потоки	3 потоки
За умовою	138.40	128.79	128.42
Рівномірний	126.94	133.92	145.10
Переважно string	158.02	162.54	168.58

Усі 9 запусків мають OK; контрольний тест: 6 операцій, checksum = 41.

6. Висновки

Для кожного потоку оброблено 300 000 операцій, тобто обсяг роботи зростає з кількістю потоків (300/600/900 тис.). Тому пряме порівняння часів не є оцінкою прискорення однакової задачі. На MacBook Pro для розподілу за умовою час зменшився зі 138,40 до 128,42 мс, а для рівномірного та переважно string зріс. Найдовше виконувалися тести з перевагою string. Результат залежить від виду навантаження.

7. Виконана робота

Реалізовано Fields, читання команд, генератор тестів без зайвих масивів, перевірку значень та запуск 1/2/3 потоків; збирання через CMake.

Важливо: усі операції з полями захищені, але через окремі дані потоків конкуренція саме за ті самі м’ютекси не виникає. Це відповідає умові про власні дані потоків й не є тестом спільного об’єкта.

Час виміряно на MacBook Pro; результати можуть відрізнятися між запусками.